SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)

Assessment Tool Weightage: 40%

Duration :60 Mins

On Class

QUESTIONS:5

Total Marks:50

Annexure A

EXPERIMENTS

Code of Conduct:

I Vishal B(192524485) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Vishal B

Q1. Syntax-Directed Definition for Evaluating Arithmetic Expressions

Aim:

To write a C program that implements a Syntax-Directed Definition (SDD) for the grammar E → E + T | T, T → T * F | F, F → num, and to evaluate the expression “3 + 5 * 2”, displaying the attribute values computed at each reduction along with the final value of the expression.

Procedure:

- Define the grammar productions and attach a synthesized attribute (val) to each non-terminal E, T and F.
- Read the arithmetic expression entered by the user as a string.
- Implement recursive functions F(), T() and E() that mirror the grammar and respect operator precedence, with F handling numbers, T handling '*', and E handling '+'.
- In F(), scan consecutive digits to form a number and return it as the synthesized attribute F.val.
- In T(), first evaluate F.val, then for every '*' encountered, evaluate the next F and multiply it into T.val (rule T → T*F).
- In E(), first evaluate T.val, then for every '+' encountered, evaluate the next T and add it into E.val (rule E → E+T).
- Print the attribute value produced at every reduction step to show how synthesized attributes are computed bottom-up.
- Display the final synthesized attribute value E.val as the result of the expression.

Program:

```
/*
 * Q1: Syntax-Directed Definition (SDD) for evaluating
arithmetic expressions
 * Grammar:
 *      E -> E + T   { E.val = E1.val + T.val }
 *      E -> T       { E.val = T.val }
 *      T -> T * F   { T.val = T1.val * F.val }
 *      T -> F       { T.val = F.val }
 *      F -> num     { F.val = num.lexval }
 *
 * All attributes above are SYNTHESIZED - they are computed
from the
```

```
 * children's attribute values as the parse tree is built
bottom-up.
 */
#include <stdio.h>
#include <ctype.h>
#include <stdlib.h>

char expr[100];
int pos = 0;

void skip_spaces() {
    while (expr[pos] == ' ') pos++;
}

int F();
int T();
int E();

/* F -> num   { F.val = num.lexval } */
int F() {
    skip_spaces();
    int val = 0;
    int start = pos;
    while (isdigit(expr[pos])) {
        val = val * 10 + (expr[pos] - '0');
        pos++;
    }
    printf("  Reduce  F -> num          : F.val = %d   (token
\"%.*s\")\n",
           val, pos - start, expr + start);
    return val;
}

/* T -> T * F | F   { T.val = T1.val * F.val  |  T.val = F.val
} */
int T() {
    int val = F();
    skip_spaces();
    while (expr[pos] == '*') {
        pos++;
        int f_val = F();
```

```
        val = val * f_val;
        printf("  Reduce  T -> T * F       : T.val = %d\n",
val);
        skip_spaces();
    }
    return val;
}

/* E -> E + T | T   { E.val = E1.val + T.val  |  E.val = T.val
} */
int E() {
    int val = T();
    skip_spaces();
    while (expr[pos] == '+') {
        pos++;
        int t_val = T();
        val = val + t_val;
        printf("  Reduce  E -> E + T       : E.val = %d\n",
val);
        skip_spaces();
    }
    return val;
}

int main() {
    printf("Enter the arithmetic expression (grammar E->E+T|T,
T->T*F|F, F->num):\n");
    fgets(expr, sizeof(expr), stdin);
    /* strip newline */
    for (int i = 0; expr[i]; i++) {
        if (expr[i] == '\n') { expr[i] = '\0'; break; }
    }

    printf("\nInput Expression: %s\n\n", expr);
    printf("Attribute evaluation trace (bottom-up, synthesized
attributes):\n");

    int result = E();

    printf("\nFinal value of the expression E.val = %d\n",
result);
```

```
    return 0;
}
```

Sample Input:

```
3 + 5 * 2
```

Output:

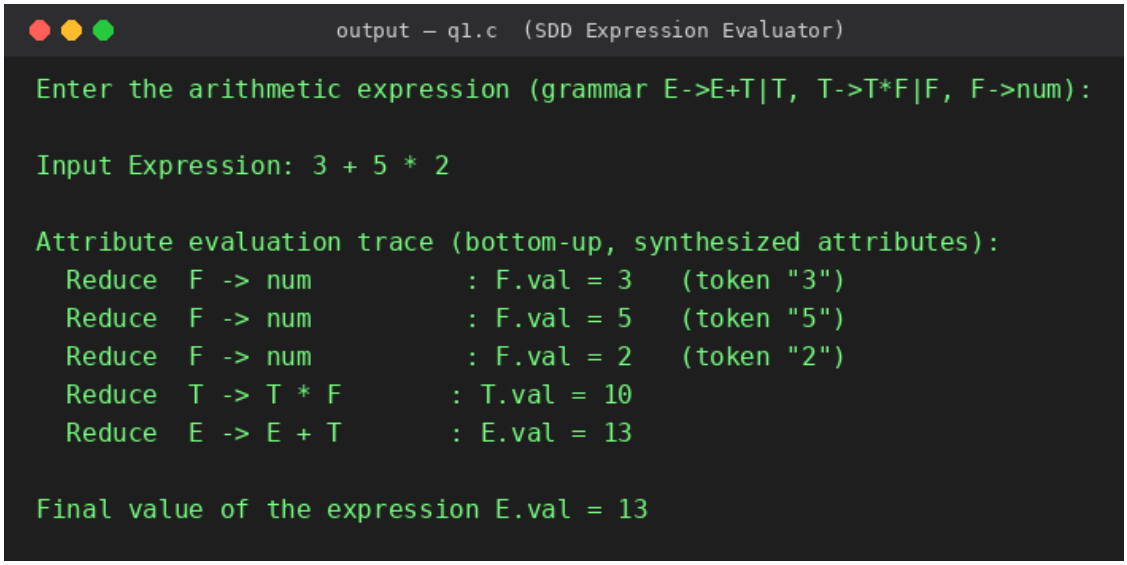
```
Enter the arithmetic expression (grammar E->E+T|T, T->T*F|F, F-
>num):

Input Expression: 3 + 5 * 2

Attribute evaluation trace (bottom-up, synthesized attributes):
  Reduce  F -> num          : F.val = 3   (token "3")
  Reduce  F -> num          : F.val = 5   (token "5")
  Reduce  F -> num          : F.val = 2   (token "2")
  Reduce  T -> T * F       : T.val = 10
  Reduce  E -> E + T       : E.val = 13

Final value of the expression E.val = 13
```

Output Screenshot:



Result:

The program was compiled and executed successfully. Following the grammar E→E+T|T, T→T*F|F, F→num, the attributes were computed bottom-up: F.val=3, F.val=5, F.val=2, then T.val=5*2=10 (T→T*F), and finally E.val=3+10=13 (E→E+T). The final evaluated value of the expression “3+5*2” is 13, confirming that synthesized attributes correctly propagate from the leaves of the parse tree to the root.

Q2. Simple Type Checker for Arithmetic Expressions

Aim:

To write a C program that implements a simple type checker for the grammar E → E + T | T, T → T * F | F, F → id, using the symbol table a:int, b:int, c:float, and to determine the type of each subexpression and the final expression “a + b * c”.

Procedure:

- Attach a synthesized attribute (type) to each non-terminal E, T and F.
- Build a symbol table that stores the declared type of every identifier: a:int, b:int, c:float.
- Implement lookup(id) to fetch the type of an identifier from the symbol table.
- Implement the semantic rule checkType(t1, t2): if either operand's type is float the result is float, otherwise it is int.
- Implement F() to read an identifier and set F.type = lookup(id.name).
- Implement T() to evaluate F.type, then for each '*' combine the running type with the next F.type using checkType() (rule T→T*F).
- Implement E() to evaluate T.type, then for each '+' combine the running type with the next T.type using checkType() (rule E→E+T).
- Print the type determined for each subexpression as it is reduced, and finally display the type of the whole expression.

Program:

```
/*
 * Q2: Simple type checker for arithmetic expressions
 * Grammar:
 *      E -> E + T   { E.type = checkType(E1.type, T.type) }
 *      E -> T       { E.type = T.type }
 *      T -> T * F   { T.type = checkType(T1.type, F.type) }
 *      T -> F       { T.type = F.type }
 *      F -> id      { F.type = lookup(id.name) }
 *
 * Rule: int op int = int ; if either operand is float => float
 * Symbol table:  a:int  b:int  c:float
 */
#include <stdio.h>
#include <string.h>
#include <ctype.h>
```

```
#define INT_T 0
#define FLOAT_T 1

typedef struct {
    char name;
    int type;
} SymEntry;

SymEntry symtab[] = { {'a', INT_T}, {'b', INT_T}, {'c',
FLOAT_T} };
int symcount = 3;

char expr[100];
int pos = 0;

const char* typeName(int t) { return t == INT_T ? "int" :
"float"; }

void skip_spaces() { while (expr[pos] == ' ') pos++; }

int lookup(char name) {
    for (int i = 0; i < symcount; i++)
        if (symtab[i].name == name) return symtab[i].type;
    printf("Error: identifier '%c' not declared\n", name);
    return INT_T;
}

int checkType(int t1, int t2) {
    return (t1 == FLOAT_T || t2 == FLOAT_T) ? FLOAT_T : INT_T;
}

int F();
int T();
int E();

/* F -> id   { F.type = lookup(id.name) } */
int F() {
    skip_spaces();
    char name = expr[pos];
    pos++;
    int t = lookup(name);
```

```
    printf("  F -> id (%c)          : type = %s\n", name,
typeName(t));
    return t;
}

/* T -> T * F | F */
int T() {
    int t = F();
    skip_spaces();
    while (expr[pos] == '*') {
        pos++;
        int t2 = F();
        t = checkType(t, t2);
        printf("  T -> T * F           : type = %s\n",
typeName(t));
        skip_spaces();
    }
    return t;
}

/* E -> E + T | T */
int E() {
    int t = T();
    skip_spaces();
    while (expr[pos] == '+') {
        pos++;
        int t2 = T();
        t = checkType(t, t2);
        printf("  E -> E + T           : type = %s\n",
typeName(t));
        skip_spaces();
    }
    return t;
}

int main() {
    printf("Symbol Table:\n");
    for (int i = 0; i < symcount; i++)
        printf("  %c : %s\n", symtab[i].name,
typeName(symtab[i].type));
```

```
    printf("\nEnter the expression (identifiers only, e.g.
a+b*c): ");
    fgets(expr, sizeof(expr), stdin);
    for (int i = 0; expr[i]; i++)
        if (expr[i] == '\n') { expr[i] = '\0'; break; }

    printf("\nInput Expression: %s\n\n", expr);
    printf("Type evaluation trace (synthesized
attributes):\n");

    int finalType = E();

    printf("\nFinal type of the expression = %s\n",
typeName(finalType));
    return 0;
}
```

Sample Input:

```
a+b*c
```

Output:

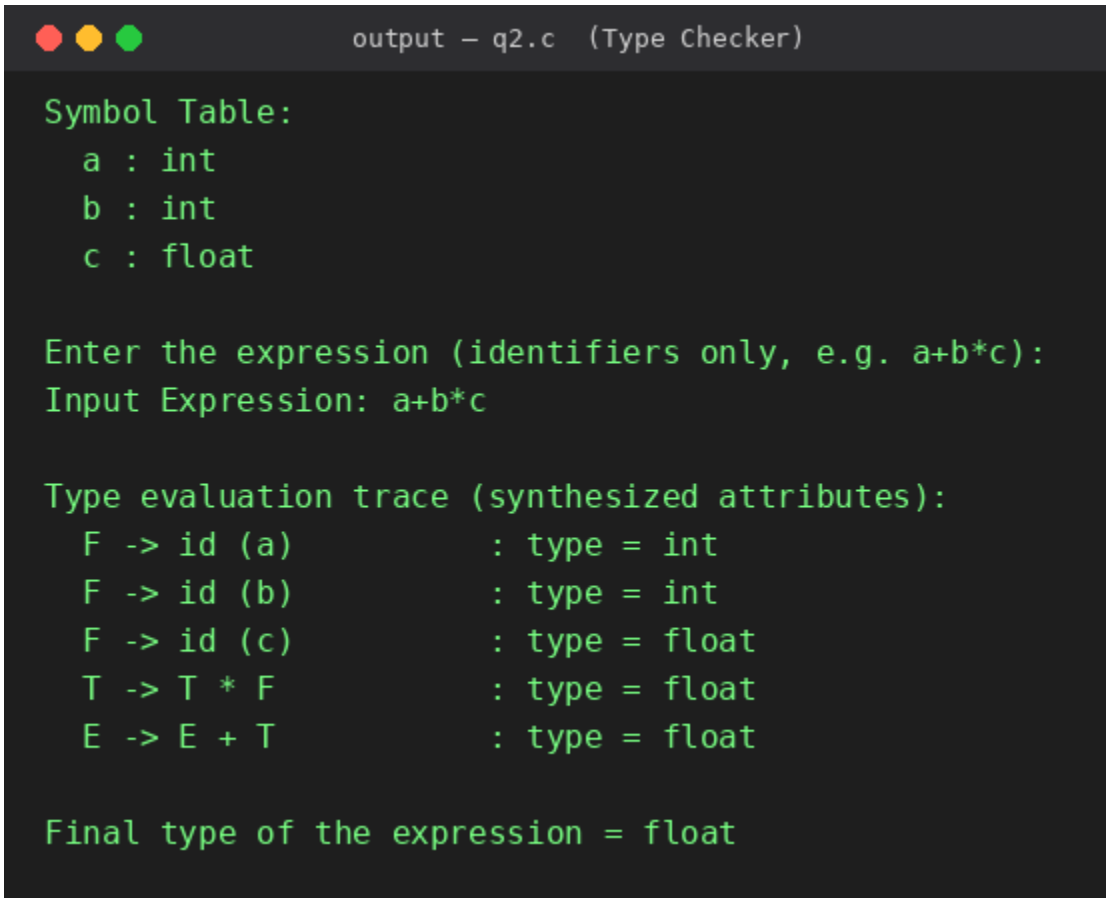
```
Symbol Table:
  a : int
  b : int
  c : float

Enter the expression (identifiers only, e.g. a+b*c):
Input Expression: a+b*c

Type evaluation trace (synthesized attributes):
  F -> id (a)          : type = int
  F -> id (b)          : type = int
  F -> id (c)          : type = float
  T -> T * F           : type = float
  E -> E + T           : type = float

Final type of the expression = float
```

Output Screenshot:



Result:

The program was compiled and executed successfully. Using the symbol table (a:int, b:int, c:float), the type checker determined F.type(a)=int, F.type(b)=int, F.type(c)=float, then T.type=checkType(int,float)=float for 'b*c', and finally E.type=checkType(int,float)=float for 'a+b*c'. The final type of the expression is float, correctly demonstrating the type-checking rule that mixing int and float promotes the result to float.

Q3. Checking Equivalence of Two Type Expressions

Aim:

To write a C program that determines whether two type expressions, T1 = array(10, int) and T2 = array(10, int), are structurally equivalent, and to display whether they are "Equivalent" or "Not Equivalent".

Procedure:

- Represent a type expression as a tree node containing a kind (BASIC / ARRAY / POINTER), a basic-type name, an array size, and a pointer to the component (child) type.
- Implement constructor functions makeBasic(name) and makeArray(size, child) to build type-expression trees such as array(10, int).
- Implement a recursive function isEquivalent(t1, t2) that returns true only when both nodes have the same kind, and — for BASIC types — the same name, or — for ARRAY types — the same size and recursively equivalent component types.
- Construct T1 = array(10, int) and T2 = array(10, int) using the constructor functions.
- Call isEquivalent(T1, T2) and print "Equivalent" or "Not Equivalent" based on the boolean result.
- As a further check, construct a differing type T3 = array(10, float) and compare it with T1 to confirm the checker correctly reports non-equivalence.

Program (C):

```
/*
 * Q3: Determine whether two type expressions are structurally
equivalent.
 * Type expressions supported:
 *      basic types : int, float, char
 *      array(size, type)
 *      pointer(type)          -- p(type)
 *
 * Represented as a simple tree; two type expressions are
equivalent
 * if their trees are structurally identical (same constructor,
 * same array size, and equivalent component types -
recursively).
 */
#include <stdio.h>
```

```
#include <stdlib.h>
#include <string.h>

typedef enum { BASIC, ARRAY, POINTER } Kind;

typedef struct TypeNode {
    Kind kind;
    char basicName[10];   /* used when kind == BASIC   e.g.
"int" */
    int size;              /* used when kind == ARRAY   e.g. 10
*/
    struct TypeNode *child; /* used when kind == ARRAY or
POINTER  */
} TypeNode;

TypeNode* makeBasic(const char *name) {
    TypeNode *t = (TypeNode*)malloc(sizeof(TypeNode));
    t->kind = BASIC;
    strcpy(t->basicName, name);
    t->child = NULL;
    return t;
}

TypeNode* makeArray(int size, TypeNode *elem) {
    TypeNode *t = (TypeNode*)malloc(sizeof(TypeNode));
    t->kind = ARRAY;
    t->size = size;
    t->child = elem;
    return t;
}

int isEquivalent(TypeNode *t1, TypeNode *t2) {
    if (t1 == NULL || t2 == NULL) return t1 == t2;
    if (t1->kind != t2->kind) return 0;

    switch (t1->kind) {
        case BASIC:
            return strcmp(t1->basicName, t2->basicName) == 0;
        case ARRAY:
            return (t1->size == t2->size) && isEquivalent(t1-
>child, t2->child);
```

```
        case POINTER:
            return isEquivalent(t1->child, t2->child);
    }
    return 0;
}

void printType(TypeNode *t) {
    if (t->kind == BASIC) {
        printf("%s", t->basicName);
    } else if (t->kind == ARRAY) {
        printf("array(%d, ", t->size);
        printType(t->child);
        printf(")");
    } else {
        printf("pointer(");
        printType(t->child);
        printf(")");
    }
}

int main() {
    /* T1 = array(10, int) */
    TypeNode *T1 = makeArray(10, makeBasic("int"));
    /* T2 = array(10, int) */
    TypeNode *T2 = makeArray(10, makeBasic("int"));

    printf("T1 = ");
    printType(T1);
    printf("\nT2 = ");
    printType(T2);
    printf("\n\n");

    if (isEquivalent(T1, T2))
        printf("Result: Equivalent\n");
    else
        printf("Result: Not Equivalent\n");

    /* Extra demonstration: a differing type expression */
    TypeNode *T3 = makeArray(10, makeBasic("float"));
    printf("\nT1 = ");
    printType(T1);
```

```
    printf("\nT3 = ");
    printType(T3);
    printf("\n\n");
    if (isEquivalent(T1, T3))
        printf("Result: Equivalent\n");
    else
        printf("Result: Not Equivalent\n");

    return 0;
}
```

Sample Input:

```
T1 = array(10, int)
T2 = array(10, int)
```

Output:

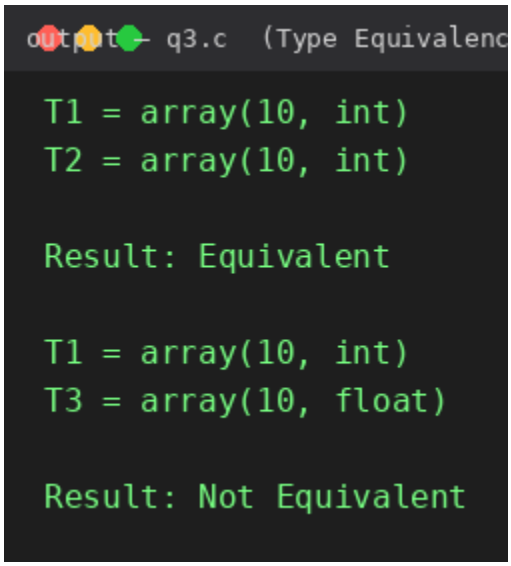
```
T1 = array(10, int)
T2 = array(10, int)

Result: Equivalent

T1 = array(10, int)
T3 = array(10, float)

Result: Not Equivalent
```

Output Screenshot:



Result:

The program was compiled and executed successfully. Comparing T1 = array(10, int) with T2 = array(10, int), the recursive structural check found identical kind (ARRAY), identical size (10) and identical component type (int), so the result printed was “Equivalent”. When T1 was compared with T3 = array(10, float), the component types differed (int vs float), so the result printed was “Not Equivalent”, confirming the correctness of the type-equivalence algorithm.

Q4. Construction and Display of the Syntax Tree

Aim:

To write a C program that constructs and displays the syntax tree for the expression “a + b * c” using the grammar E → E + T | T, T → T * F | F, F → id.

Procedure:

- Define a tree node structure Node containing an operator/identifier field and left and right child pointers.
- Implement mkleaf(name) to create a leaf node for an identifier (used in rule F→id).
- Implement mknode(op, left, right) to create an internal node representing an operator with two subtrees (used in rules E→E+T and T→T*F).
- Implement recursive-descent functions F(), T() and E() that parse the input string and, following the grammar, build the syntax tree bottom-up: each rule returns a node pointer as its synthesized attribute.
- In T(), combine the current node and the next F using mknode('*', ...) whenever '*' is seen; in E(), combine using mknode('+', ...) whenever '+' is seen.
- Implement a traversal function to display the constructed tree in an indented layout, and another to print it in prefix (Polish) notation for verification.
- Call E() on the input “a+b*c” and display the resulting syntax tree.

Program:

```
/*
 * Q4: Construct and display the syntax tree for an expression
 * Grammar:
 *      E -> E + T   { E.node = mknode('+', E1.node, T.node) }
 *      E -> T       { E.node = T.node }
 *      T -> T * F   { T.node = mknode('*', T1.node, F.node) }
 *      T -> F       { T.node = F.node }
 *      F -> id      { F.node = mkleaf(id.name) }
 *
 * The tree is built bottom-up using synthesized attributes
(node pointers)
 * and then displayed both as an indented tree and in prefix
(Polish) form.
 */
#include <stdio.h>
#include <stdlib.h>
```

```
typedef struct Node {
    char op;              /* operator, or the identifier if
leaf */
    int isLeaf;
    struct Node *left, *right;
} Node;

char expr[100];
int pos = 0;

Node* mkleaf(char name) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->op = name;
    n->isLeaf = 1;
    n->left = n->right = NULL;
    return n;
}

Node* mknode(char op, Node *l, Node *r) {
    Node *n = (Node*)malloc(sizeof(Node));
    n->op = op;
    n->isLeaf = 0;
    n->left = l;
    n->right = r;
    return n;
}

void skip_spaces() { while (expr[pos] == ' ') pos++; }

Node* F();
Node* T();
Node* E();

/* F -> id */
Node* F() {
    skip_spaces();
    char name = expr[pos++];
    return mkleaf(name);
}
```

```
/* T -> T * F | F */
Node* T() {
    Node *n = F();
    skip_spaces();
    while (expr[pos] == '*') {
        pos++;
        Node *r = F();
        n = mknode('*', n, r);
        skip_spaces();
    }
    return n;
}

/* E -> E + T | T */
Node* E() {
    Node *n = T();
    skip_spaces();
    while (expr[pos] == '+') {
        pos++;
        Node *r = T();
        n = mknode('+', n, r);
        skip_spaces();
    }
    return n;
}

/* Display as an indented tree (root at left, children
indented) */
void printTree(Node *n, int depth) {
    if (!n) return;
    printTree(n->right, depth + 1);
    for (int i = 0; i < depth; i++) printf("    ");
    printf("%c\n", n->op);
    printTree(n->left, depth + 1);
}

/* Prefix (Polish) notation traversal */
void printPrefix(Node *n) {
    if (!n) return;
    printf("%c", n->op);
```

```
    if (!n->isLeaf) {
        printf("(");
        printPrefix(n->left);
        printf(",");
        printPrefix(n->right);
        printf(")");
    }
}

int main() {
    printf("Enter expression (identifiers a-z, + and * only,
e.g. a+b*c): ");
    fgets(expr, sizeof(expr), stdin);
    for (int i = 0; expr[i]; i++)
        if (expr[i] == '\n') { expr[i] = '\0'; break; }

    printf("\nInput Expression: %s\n\n", expr);

    Node *root = E();

    printf("Syntax Tree (rotated 90 deg - root on left, leaves
on right):\n\n");
    printTree(root, 0);

    printf("\nPrefix form of the syntax tree: ");
    printPrefix(root);
    printf("\n");

    return 0;
}
```

Sample Input:

```
a+b*c
```

Output:

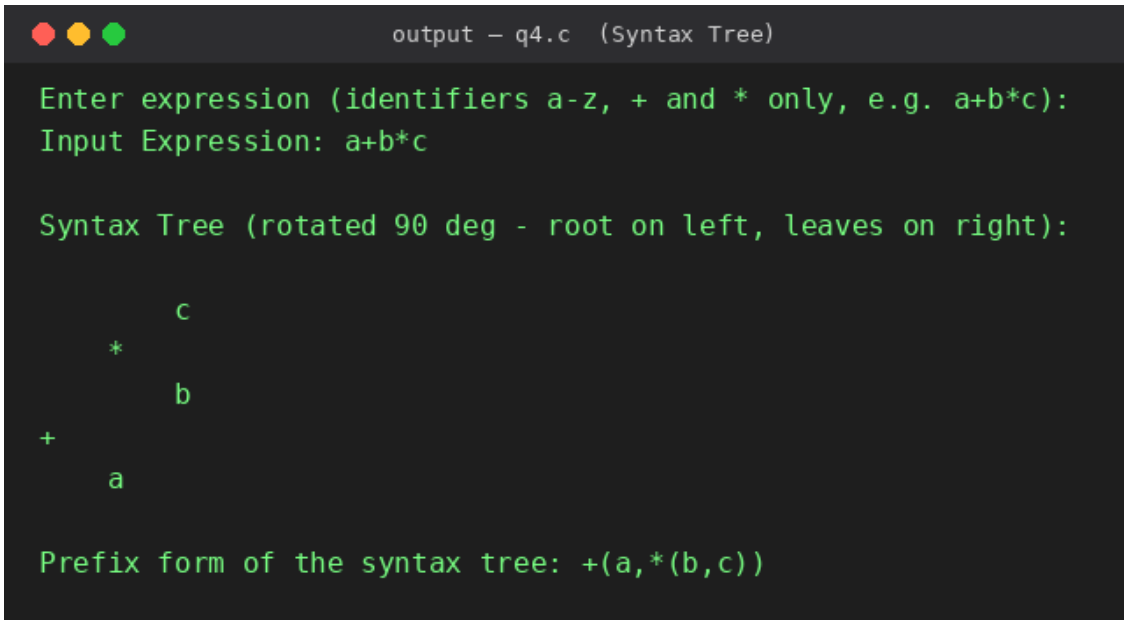
```
Enter expression (identifiers a-z, + and * only, e.g. a+b*c):
Input Expression: a+b*c
```

```
Syntax Tree (rotated 90 deg - root on left, leaves on right):

        c
    *
        b
+
    a

Prefix form of the syntax tree: +(a,*(b,c))
```

Output Screenshot:



Result:

The program was compiled and executed successfully. For the input “a+b*c”, the syntax tree was correctly constructed with '+' as the root node, 'a' as its left child, and the subtree '*' (with children 'b' and 'c') as its right child — matching operator precedence (multiplication binds tighter than addition). The indented tree display and the prefix form “+(a,*(b,c))” both confirm the correct tree structure.

Q5. Propagation of Inherited and Synthesized Attributes

Aim:

To write a C program that demonstrates the propagation of inherited and synthesized attributes using the grammar D → T L, T → int, L → id L', L' → , id L' | ε, and to display the type assigned to each identifier for the declaration “int a,b,c”.

Procedure:

- Identify T.type as a SYNTHESIZED attribute (computed at T→int) and L.inh / L'.inh as INHERITED attributes that carry the declared type down from D to every identifier in the list.
- Implement T_rule() to recognize the keyword “int” in the input and return the synthesized attribute type = “int”.
- Implement D() to call T_rule() first, then pass its synthesized result down as the inherited attribute L.inh (rule D→T L).
- Implement L(inh) to read the first identifier, record it with the inherited type using addType(), and pass the same inherited attribute down to L'() (rule L→id L').
- Implement L'(inh) recursively: if a comma is found, read the next identifier, record it with the inherited type inh, and recurse with the same inh (rule L'→,id L'); if no comma is found, stop (rule L'→ε).
- Maintain a symbol table updated by addType(name, type) each time an identifier is processed.
- After parsing the full declaration, print the final symbol table showing the type assigned to every identifier.

Program :

```
/*
 * Q5: Propagation of inherited and synthesized attributes
 * Grammar:
 *      D  -> T L        { L.inh = T.type }
(T.type is SYNTHESIZED,
 *
L.inh is INHERITED from D)
 *      T  -> int        { T.type = "int" }
(synthesized)
 *      L  -> id L'      { addType(id.name, L.inh)
 *                          L'.inh = L.inh }
(L.inh inherited down to L')
 *      L' -> , id L'1   { addType(id.name, L'.inh)
```

```
 *                           L'1.inh = L'.inh }
 *      L' -> eps
 *
 * L.inh and L'.inh are INHERITED attributes: the type flows
DOWN the
 * parse tree from the declaration keyword (T) to every
identifier in
 * the list. Nothing is computed bottom-up here except T.type
itself.
 *
 * Input: int a,b,c
 */
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAXVARS 20

char names[MAXVARS][20];
char types[MAXVARS][20];
int count = 0;

char input[100];
int pos = 0;

void skip_spaces() { while (input[pos] == ' ') pos++; }

void addType(const char *name, const char *type) {
    strcpy(names[count], name);
    strcpy(types[count], type);
    printf("  addType(\"%s\", inherited type = \"%s\")\n",
name, type);
    count++;
}

/* T -> int   { T.type = "int" }   -- SYNTHESIZED */
char* T_rule() {
    skip_spaces();
    /* consume the literal "int" */
    pos += 3;
```

```
    printf("  T -> int              : T.type = \"int\"
(synthesized)\n");
    return "int";
}

void readIdent(char *buf) {
    skip_spaces();
    int i = 0;
    while (isalnum(input[pos])) buf[i++] = input[pos++];
    buf[i] = '\0';
}

/* L' -> , id L'   |   eps      -- receives inherited attribute
'inh' */
void Lprime(const char *inh) {
    skip_spaces();
    if (input[pos] == ',') {
        pos++;
        char id[20];
        readIdent(id);
        printf("  L' -> , id L'         : L'.inh passed down =
\"%s\"\n", inh);
        addType(id, inh);
        Lprime(inh);   /* inherited attribute passed unchanged
to next L' */
    } else {
        printf("  L' -> eps             : end of identifier
list\n");
    }
}

/* L -> id L'      -- receives inherited attribute 'inh' from D
-> T L */
void L(const char *inh) {
    char id[20];
    readIdent(id);
    printf("  L -> id L'            : L.inh received =
\"%s\"\n", inh);
    addType(id, inh);
    Lprime(inh);
```

```
}

/* D -> T L */
void D() {
    char *type = T_rule();      /* synthesized attribute from T
*/
    L(type);                    /* type is passed DOWN as L.inh
*/
}

int main() {
    printf("Enter declaration (e.g. int a,b,c): ");
    fgets(input, sizeof(input), stdin);
    for (int i = 0; input[i]; i++)
        if (input[i] == '\n') { input[i] = '\0'; break; }

    printf("\nInput: %s\n\n", input);
    printf("Attribute propagation trace:\n");

    D();

    printf("\nFinal Symbol Table (type assigned to each
identifier):\n");
    printf("  %-10s %-10s\n", "Identifier", "Type");
    for (int i = 0; i < count; i++)
        printf("  %-10s %-10s\n", names[i], types[i]);

    return 0;
}
```

Sample Input:

```
int a,b,c
```

Output:

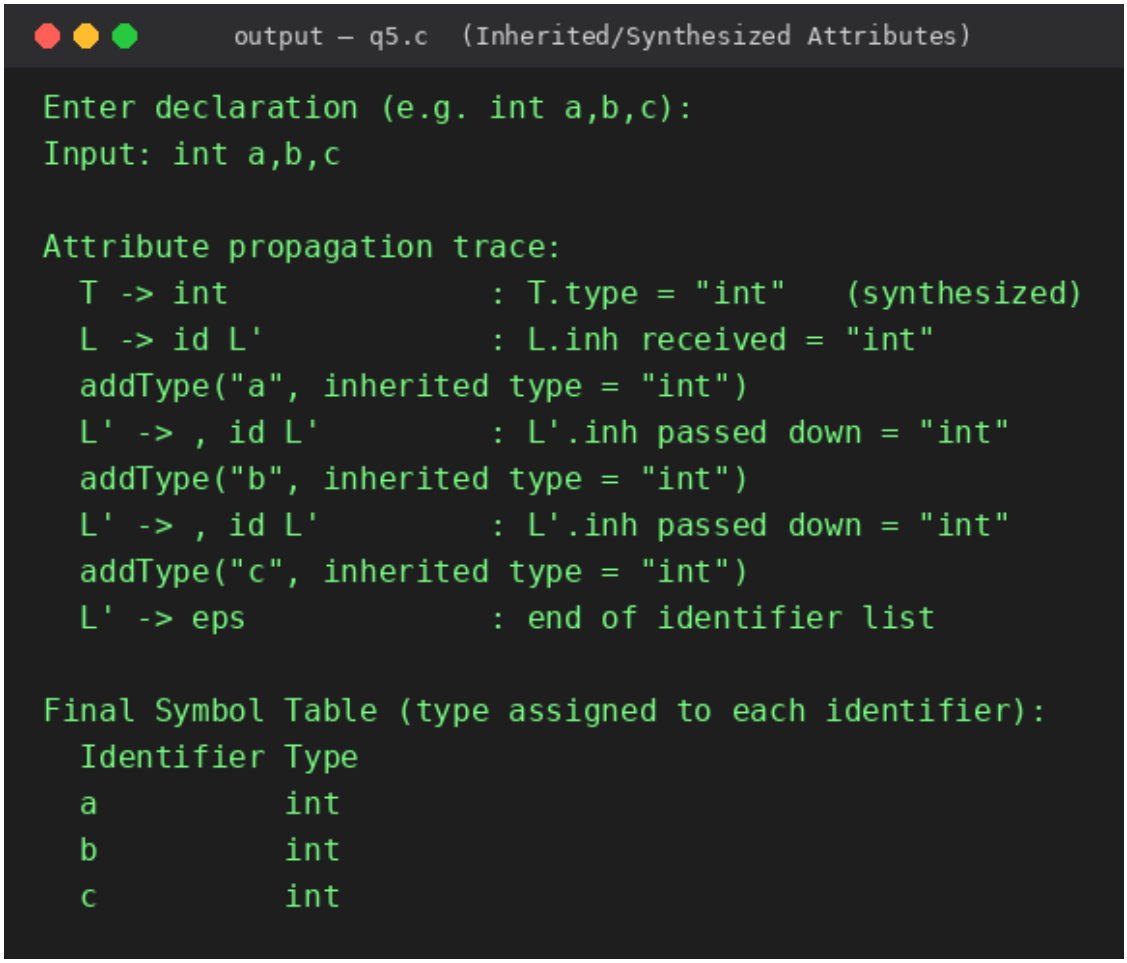
```
Enter declaration (e.g. int a,b,c):
Input: int a,b,c

Attribute propagation trace:
```

```
  T -> int              : T.type = "int"   (synthesized)
  L -> id L'            : L.inh received = "int"
  addType("a", inherited type = "int")
  L' -> , id L'         : L'.inh passed down = "int"
  addType("b", inherited type = "int")
  L' -> , id L'         : L'.inh passed down = "int"
  addType("c", inherited type = "int")
  L' -> eps             : end of identifier list

Final Symbol Table (type assigned to each identifier):
  Identifier Type
  a          int
  b          int
  c          int
```

Output Screenshot:



Result:

The program was compiled and executed successfully. For the input “int a,b,c”, T→int first produced the synthesized attribute T.type=“int”. This value was then passed down as the inherited attribute L.inh to L, and further down through each L'→,id L' application to L'.inh, so that every identifier (a, b and c) inherited the same type “int” from the declaration. The final symbol table correctly lists a:int, b:int and c:int, demonstrating top-down (inherited) attribute propagation combined with the bottom-up (synthesized) attribute from T.